

Modelado a partir de Datos Discretos

Polinomios de Interpolación de Lagrange y
Diferencias Divididas

Modelado y Simulación | Clase 3

Ing. Omar Cáceres

El Mundo nos da Puntos, No Funciones

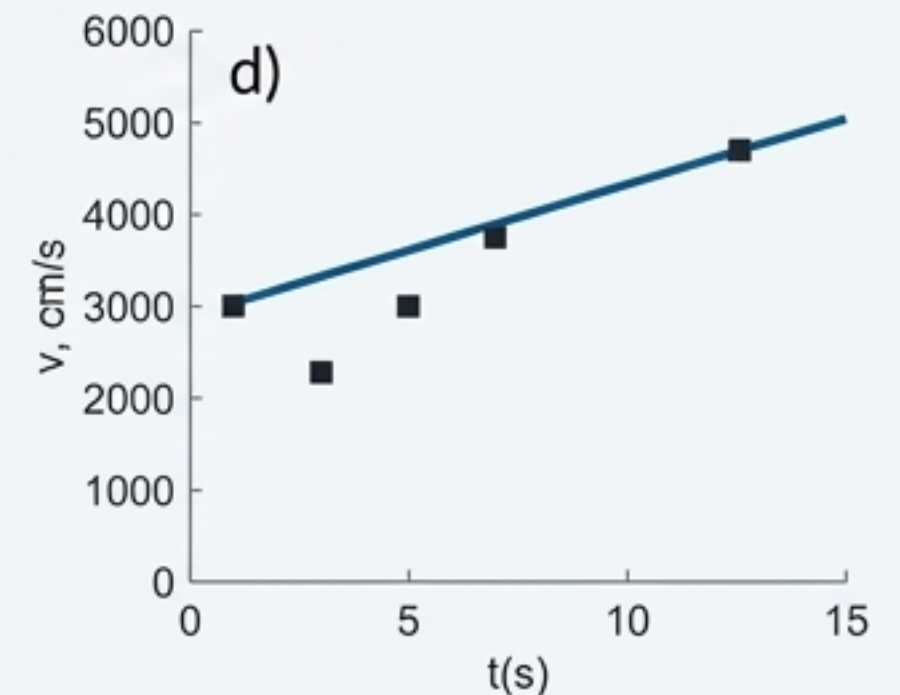
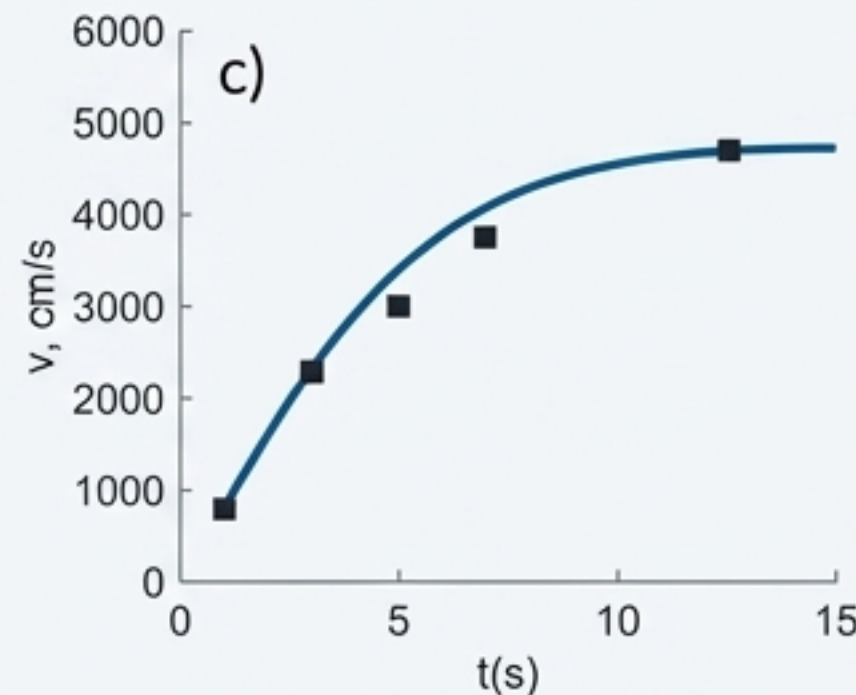
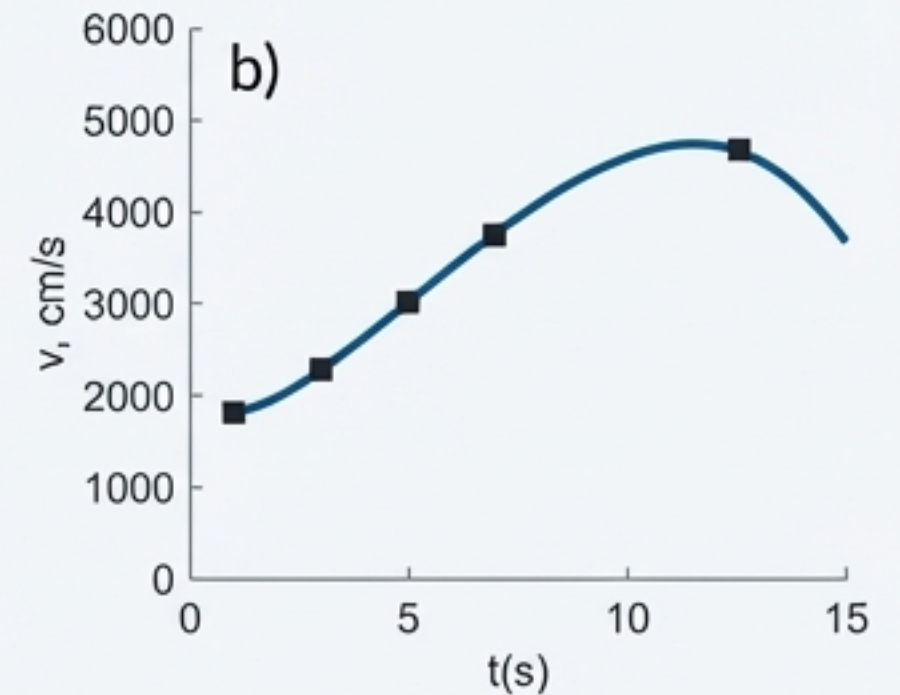
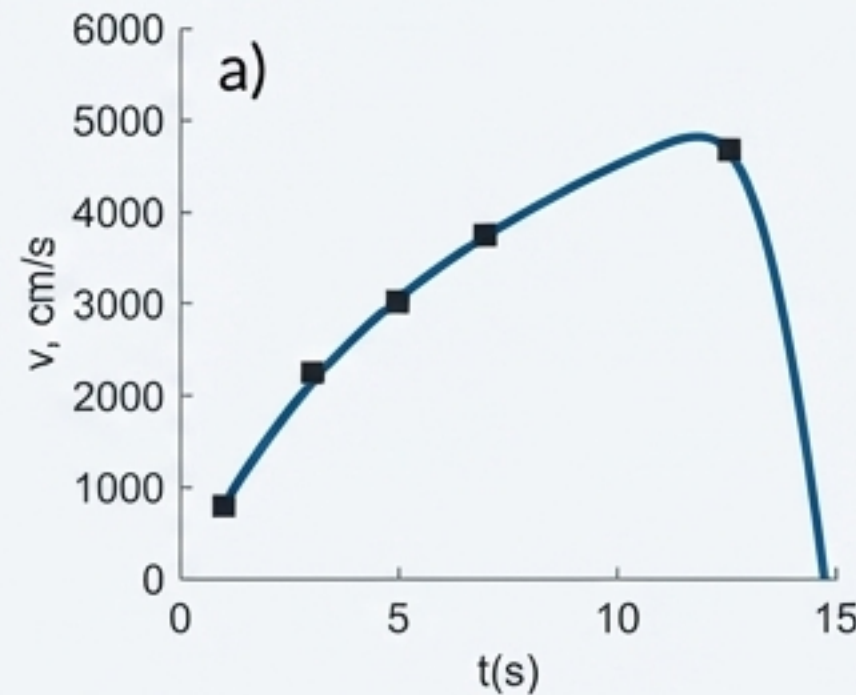
En la ciencia y la ingeniería, rara vez trabajamos con funciones perfectas y continuas. Lo que obtenemos de experimentos, sensores o mediciones es un conjunto de puntos de datos discretos.

El desafío fundamental es: ¿cómo modelamos el sistema continuo subyacente a partir de esta información limitada?

Key Insight

El Problema Central

Pasar de datos discretos a un modelo funcional continuo es uno de los retos más comunes y cruciales en el modelado numérico.



Construyendo un Puente Entre los Puntos: El Polinomio de Lagrange

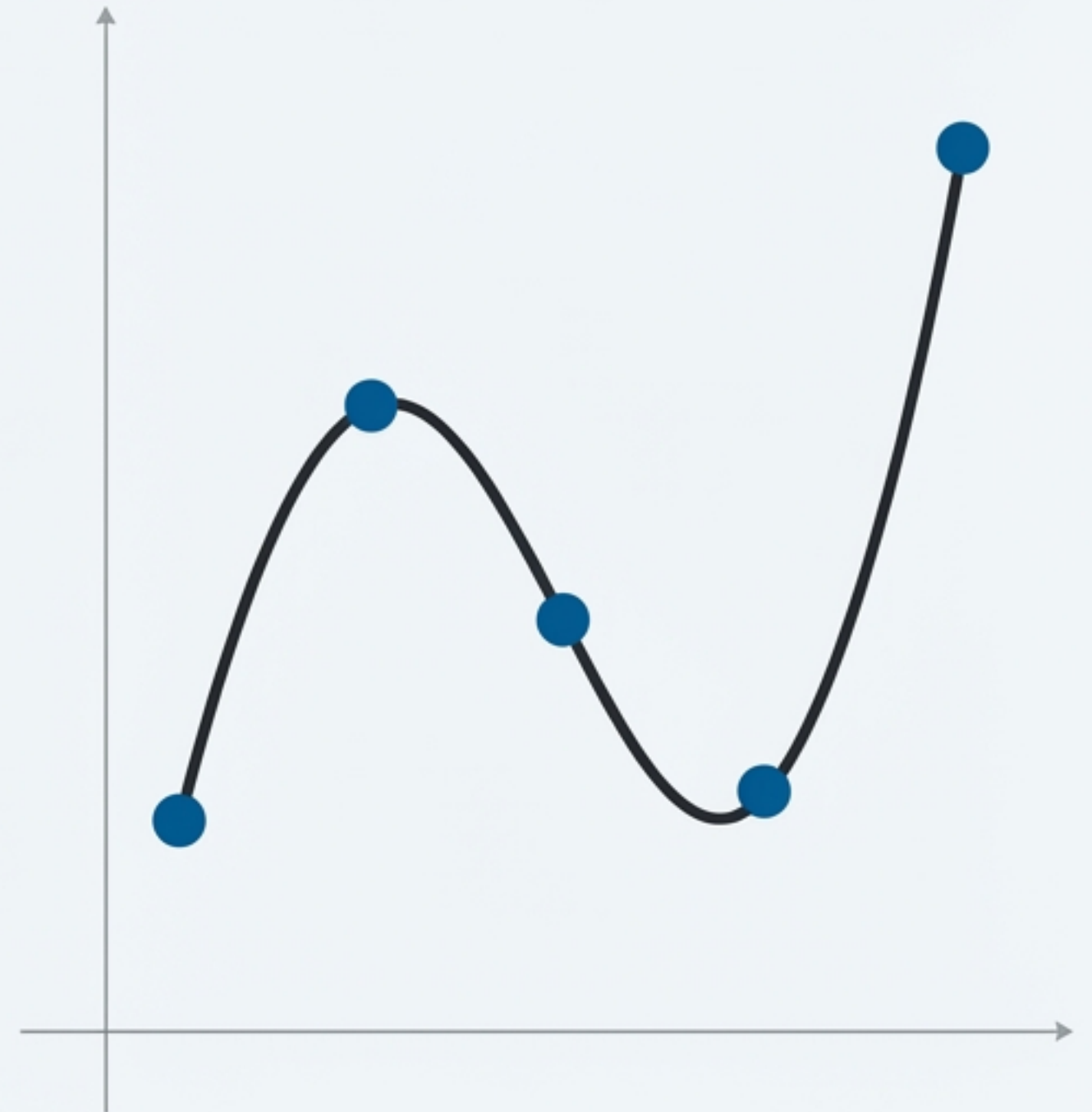
Definición:

El **polinomio interpolante de Lagrange** es una técnica matemática que construye un polinomio único que pasa *exactamente* por un conjunto dado de puntos distintos. No es una aproximación que busca "acercarse" a los puntos; es una reconstrucción precisa que los contiene.

$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

Explicación:

Este polinomio es una suma ponderada. Cada punto y_i es ponderado por un "polinomio base" $L_i(x)$, diseñado específicamente para "activar" la influencia del punto y_i en la región correcta.



La Clave del Método: Los Polinomios Base de Lagrange

La genialidad del método de Lagrange reside en la construcción de sus polinomios base, $L_i(x)$. Cada $L_i(x)$ está diseñado con dos propiedades cruciales:

1. Toma el valor de **1** en su punto correspondiente x_i .
2. Toma el valor de **0** en todos los demás puntos de datos x_j (donde $j \neq i$).

$$L_i(x) = \prod_{j=0, i \neq j}^n \frac{x - x_j}{x_i - x_j}$$

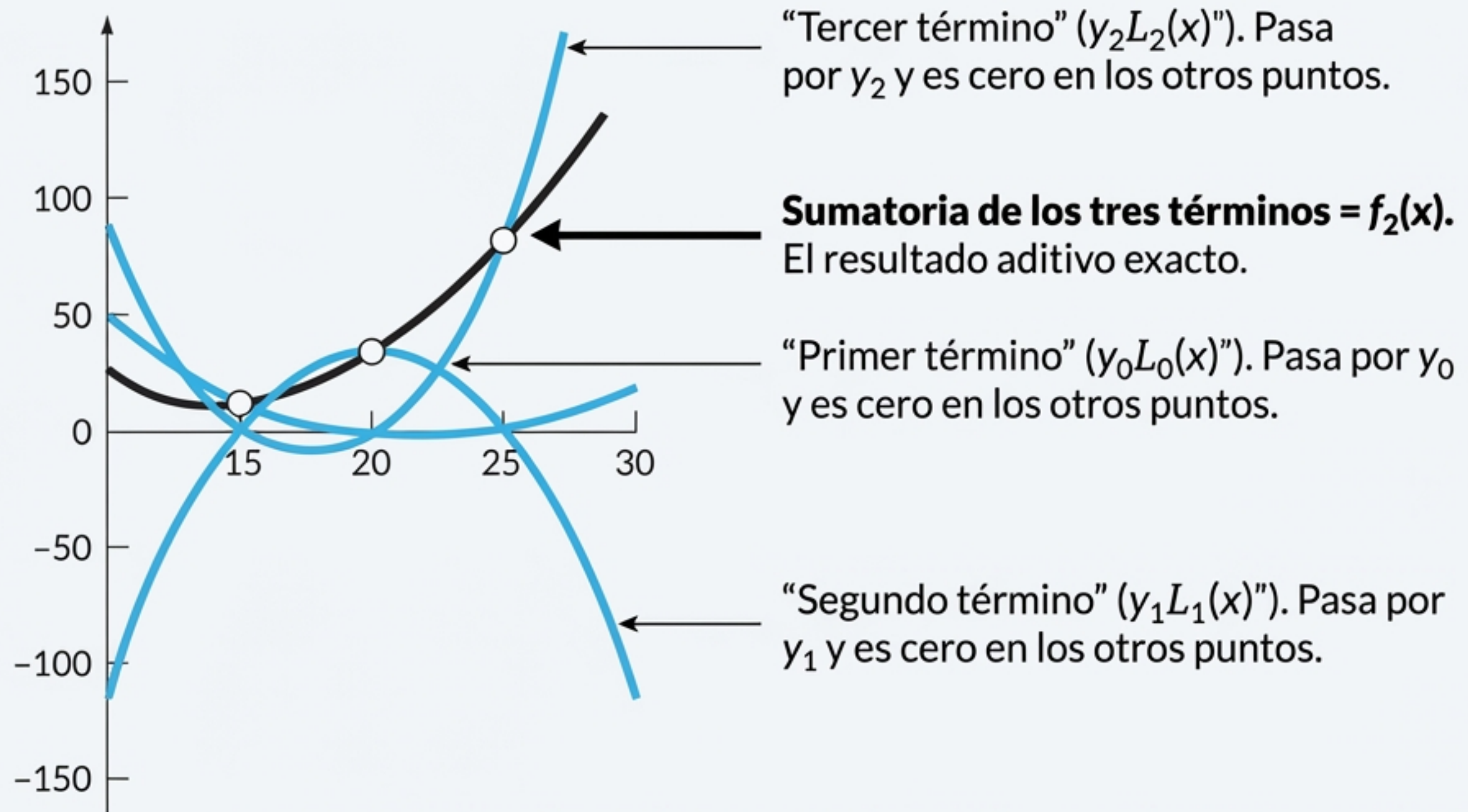
Asegura que el polinomio sea cero en todos los x_j (excepto x_i).

Es un término de normalización que asegura que el polinomio valga uno en x_i .

Key Insight: Interruptores Matemáticos

Piense en cada $L_i(x)$ como un “interruptor”. Se “enciende” (vale 1) en x_i para incluir la contribución de y_i , y se “apaga” (vale 0) en los demás puntos para no interferir.

Viendo Cómo Cada Término Construye la Curva Final



La curva final (en negro) no es una curva arbitraria; es la suma matemática precisa de los polinomios base ponderados. Esta visualización demuestra cómo la construcción de Lagrange garantiza que el polinomio final pase por cada punto de datos.

Existencia y Unicidad: La Certeza del Modelo

El método de Lagrange no es una heurística; se basa en una garantía matemática fundamental.

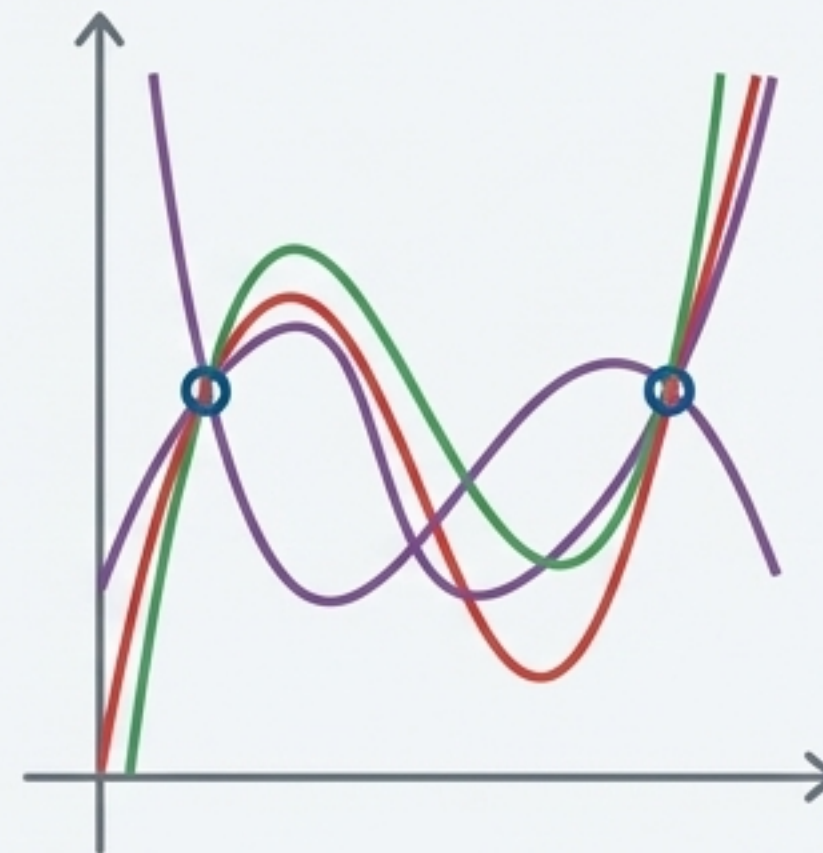
Dos Garantías Clave

1. **Existencia:** Siempre es posible construir un polinomio de interpolación de Lagrange para cualquier conjunto de $n+1$ puntos distintos.
2. **Unicidad:** Para ese conjunto de puntos, solo existe un polinomio de grado n que pasa exactamente por todos ellos. Esto está garantizado por el teorema fundamental de la interpolación.

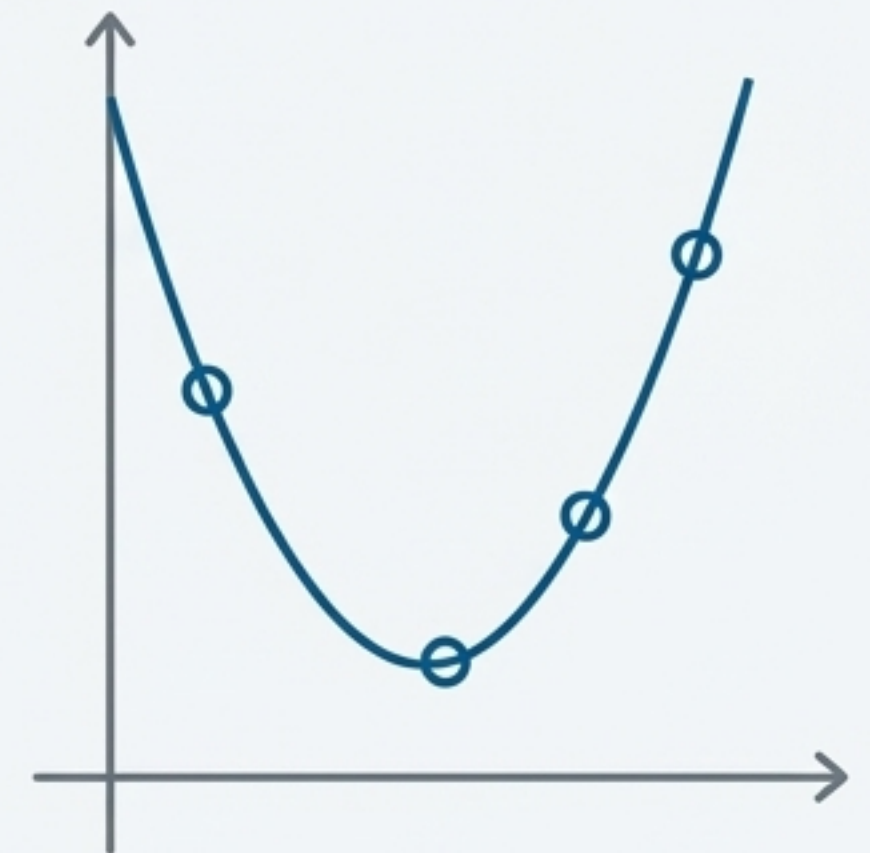
Implicación Práctica

Esta unicidad significa que el modelo que construimos no es arbitrario. Es el polinomio de grado más bajo que describe perfectamente nuestros datos. No hay ambigüedad.

Múltiples posibilidades

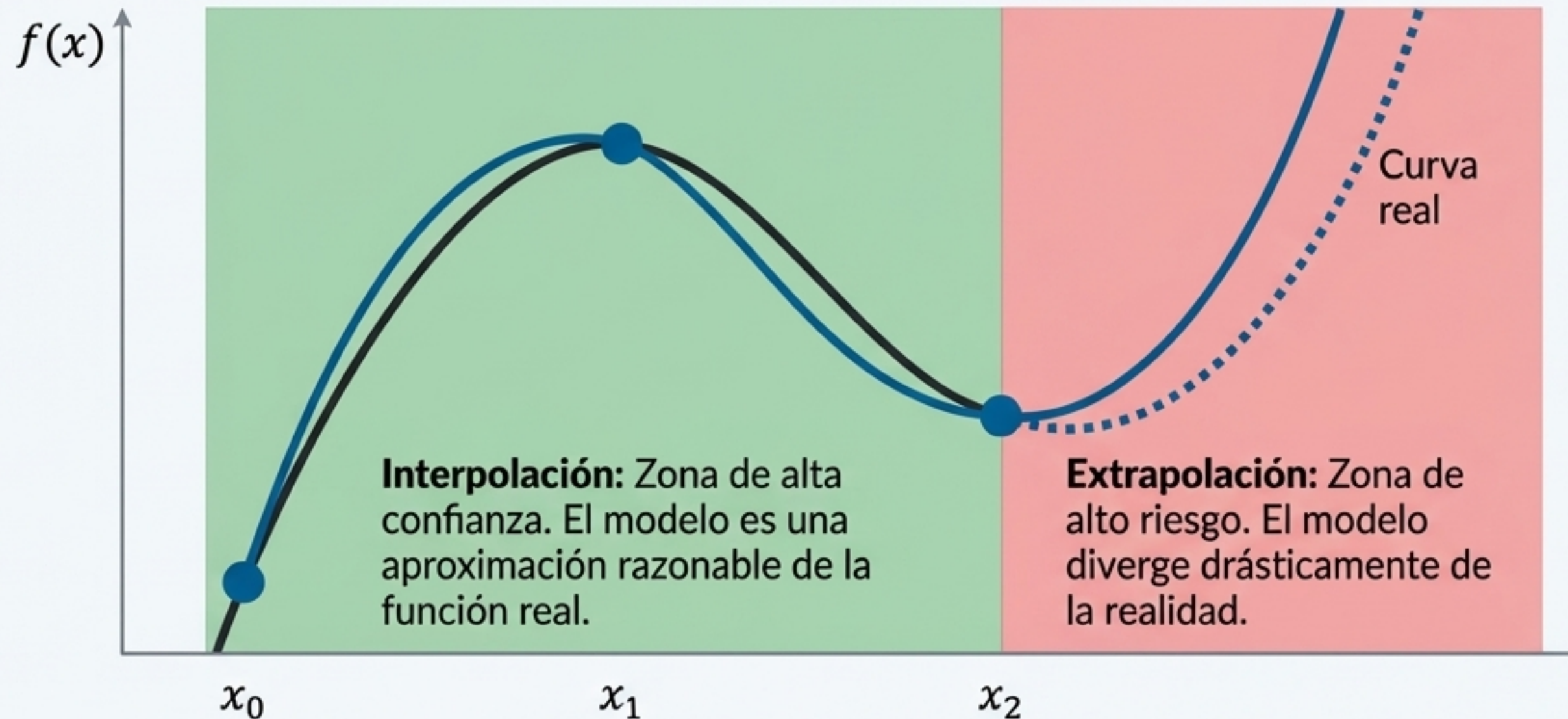


Solución Única



Los Límites del Modelo: Interpolación vs. Extrapolación

Nuestro polinomio es una representación perfecta de los datos que tenemos. Pero, ¿qué tan confiable es para predecir valores entre los puntos o más allá de ellos?



Key Insight:

Extrapolar con polinomios de alto grado es peligroso. El modelo está optimizado para los datos que conoce, no para lo que desconoce.

Advertencia de Ingeniería

¿Qué Tan Precisa es Nuestra Aproximación? El Error de Interpolación

La diferencia entre la función real $f(x)$ y nuestro polinomio interpolante $P(x)$ se conoce como el **error de interpolación**. Afortunadamente, no tenemos que adivinar su magnitud; podemos acotarla.

$$f(x) - P(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

$f^{(n+1)}(\xi)$: La $(n+1)$ -ésima derivada de la función real. Mide cuán ‘curva’ o compleja es la función real. Funciones más simples tienen derivadas más pequeñas y, por lo tanto, menos error.

$(n+1)!$: El factorial. Crece rápidamente, ayudando a reducir el error a medida que usamos más puntos (n) .

$\prod (x - x_i)$: El producto de las distancias. El error es cero en los puntos de datos ($x = x_i$) y es más grande entre ellos.

Calculando la Cota de Error: Un Proceso Sistemático

Para encontrar el error máximo posible en un intervalo, seguimos estos pasos:

- 1. Construir el Polinomio Interpolante $P(x)$**
El primer paso es tener el modelo.
- 2. Calcular el Término del Producto $\prod(x - x_i)$**
Analizar cómo se comporta este término en el intervalo de interés.
- 3. Determinar $M_{-(n+1)}$**
Encontrar el valor máximo absoluto de la $(n+1)$ -ésima derivada en el intervalo. Este es a menudo el paso más desafiante. $M_{-(n+1)} = \max|f^{(n+1)}(\xi)|$.
- 4. Aplicar la Cota**
Combinar todo en la fórmula de la cota de error. $|E(x)| \leq \frac{M_{n+1}}{(n+1)!} \left| \prod_{i=0}^n (x - x_i) \right|$.
- 5. (Opcional) Verificar con el Error Real**
Si se conoce la función $f(x)$, se puede calcular el error real $E(x) = f(x) - P(x)$ y comparar con la cota.

Expandiendo el Arsenal: De Interpolar Valores a Estimar Derivadas

Pregunta Clave

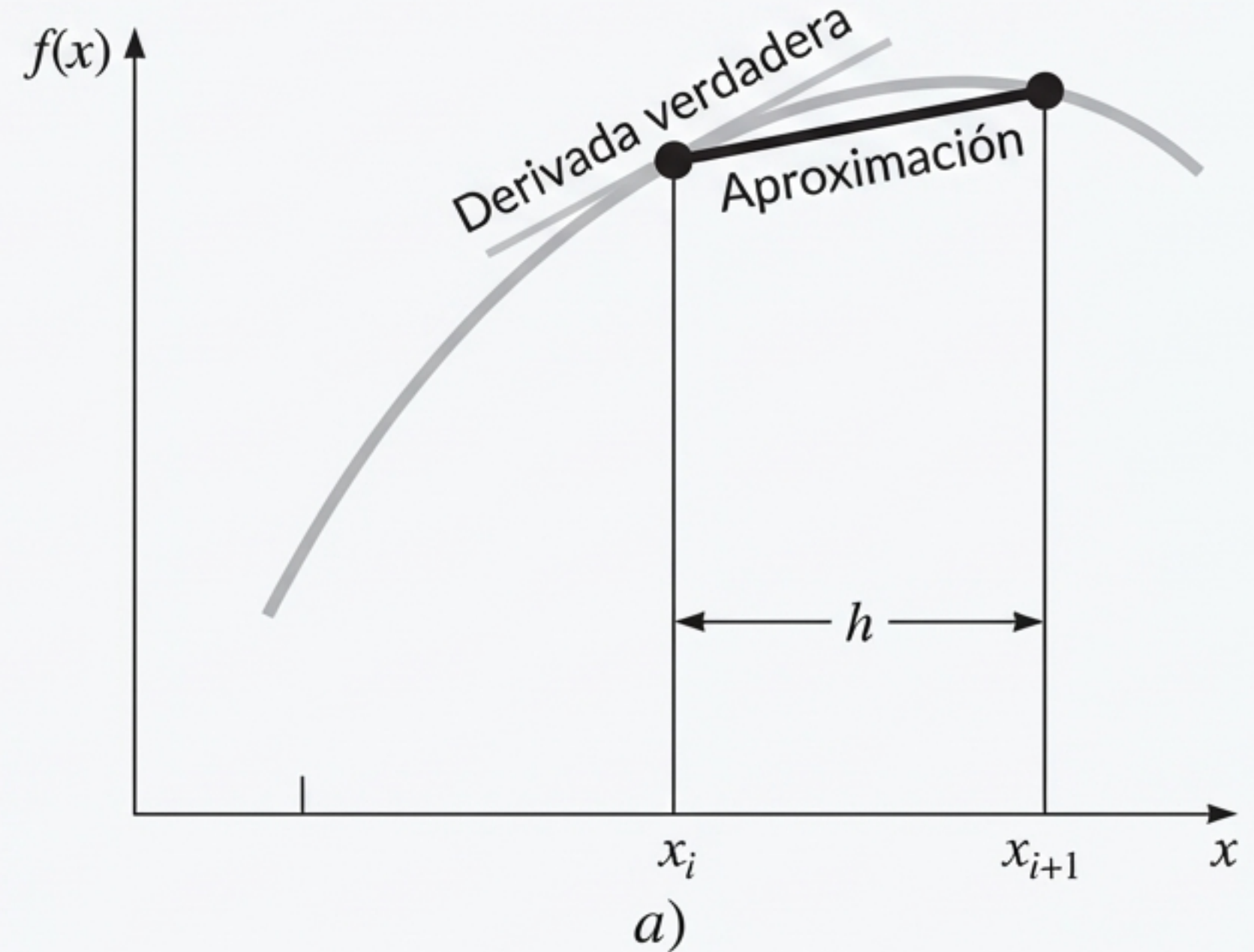
Tenemos un conjunto de puntos (x_i, y_i) . Ya sabemos cómo encontrar y para cualquier x intermedio. Pero, ¿cómo podemos estimar la *tasa de cambio* (la derivada) en esos puntos sin conocer la función subyacente?

La Conexión Lógica

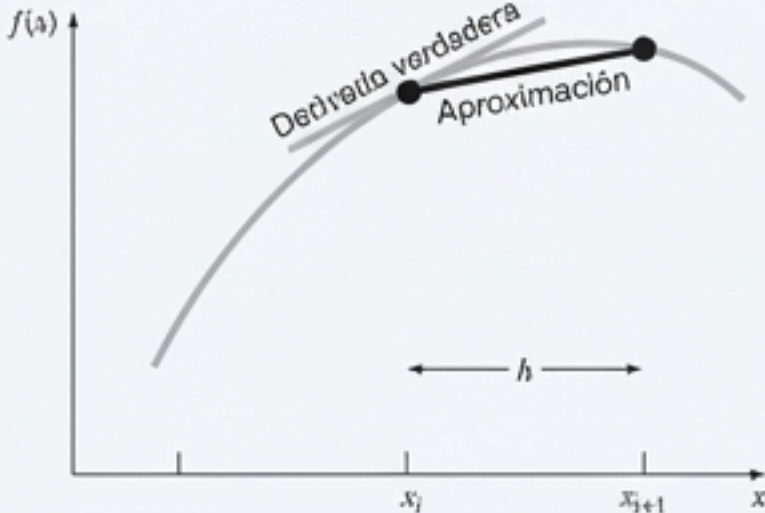
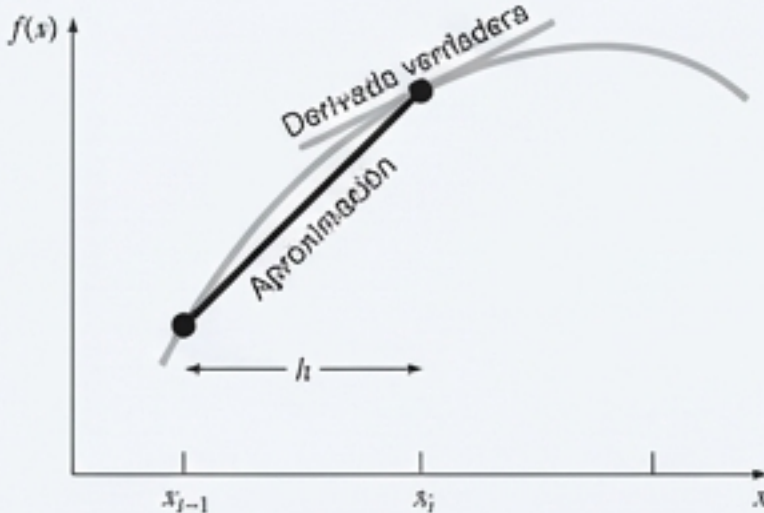
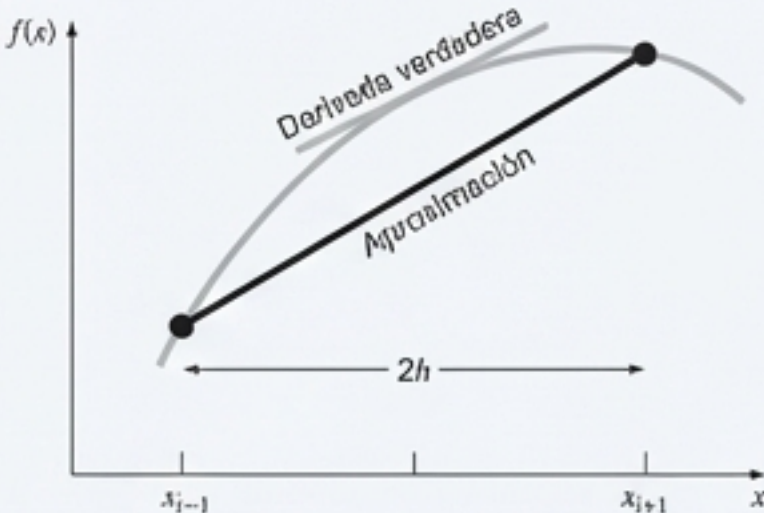
La derivada es el límite de la pendiente de una secante. Si tenemos puntos cercanos, podemos usar la pendiente de la línea que los une como una aproximación de la pendiente de la tangente (la derivada).

El Siguiente Nivel de Modelado

Estimar derivadas a partir de datos discretos es fundamental para analizar la dinámica de un sistema: velocidades, aceleraciones, tasas de reacción, etc.



Tres Enfoques para la Derivación Numérica

Diferencias Progresivas (Hacia Adelante)	Diferencias Regresivas (Hacia Atrás)	Diferencias Centrales
		
Concepto Usa el punto actual (x_i) y el siguiente (x_{i+1}) para estimar la pendiente.	Concepto Usa el punto actual (x_i) y el anterior (x_{i-1}) para estimar la pendiente.	Concepto Usa los puntos a ambos lados (x_{i-1} , x_{i+1}). Generalmente más preciso.
1ª Derivada $f'(x_i) \approx \frac{f(x_{i+1}) - f(x_i)}{h}$	1ª Derivada $f'(x_i) \approx \frac{f(x_i) - f(x_{i-1}))}{h}$	1ª Derivada $f'(x_i) \approx \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}$
2ª Derivada $f''(x_i) \approx \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2}$	2ª Derivada $f''(x_i) \approx \frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2}))}{h^2}$	2ª Derivada $f''(x_i) \approx \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2}$
Uso Ideal En el punto inicial de un conjunto de datos.	Uso Ideal En el punto final de un conjunto de datos.	Uso Ideal Para todos los puntos intermedios.

De la Fórmula al Código: Implementando Lagrange en Python

La elegancia del método de Lagrange se traduce directamente en código. A continuación se muestra una implementación en Python usando `numpy` y `matplotlib`.

```
import numpy as np
import matplotlib.pyplot as plt

def polinomio_lagrange(x, x_puntos, y_puntos):
    n = len(x_puntos)
    L = 0
    for i in range(n):
        # Calcular el polinomio base de Lagrange
        li = 1
        for j in range(n):
            if i != j:
                li *= (x - x_puntos[j]) / (x_puntos[i] - x_puntos[j])
        L += y_puntos[i] * li
    return L

# Definir los puntos dados
x_puntos = np.array([0, 1, 2, 3, 4])
y_puntos = np.array([1, 2, 0, 2, 3])

# Crear un conjunto de valores x para graficar
x = np.linspace(min(x_puntos) - 1, max(x_puntos) + 1, 400)
y = [polinomio_lagrange(xi, x_puntos, y_puntos) for xi in x]

# Graficar
plt.scatter(x_puntos, y_puntos, color='red', label='Puntos dados')
plt.plot(x, y, label='Polinomio de Lagrange')
plt.legend()
plt.grid(True)
plt.show()
```

Bucle Principal

El bucle externo (**for i**) itera sobre cada punto de datos para construir la suma $\sum y_i * L_i(x)$.

Construcción de la Base

El bucle interno (**for j**) construye el polinomio base $L_i(x)$ calculando el producto $\prod (x - x_j) / (x_i - x_j)$.

Visualización

El resto del código simplemente define los datos y utiliza la función para generar y graficar el polinomio resultante.

De la Aproximación al Código: Implementando Diferencias Divididas

Calcular derivadas numéricamente es un proceso directo en código. Aquí, implementamos las fórmulas de diferencias finitas centradas, que suelen ofrecer la mejor precisión.

```
# Definir la funcion
def f(x):
    return x**3 - 2*x + 1

# Punto y paso
x = 2
h = 0.1

# Diferencias finitas centradas para la primera derivada
def primera_derivada(f, x, h):
    return (f(x + h) - f(x - h)) / (2 * h)

# Diferencias finitas centradas para la segunda derivada
def segunda_derivada(f, x, h):
    return (f(x + h) - 2*f(x) + f(x - h)) / (h**2)

# Calcular y mostrar las derivadas
primera = primera_derivada(f, x, h)
segunda = segunda_derivada(f, x, h)

print(f"Primera derivada en x = {x}: {primera}")
print(f"Segunda derivada en x = {x}: {segunda}")
```

Traducción Directa

La función `primera_derivada` es una traducción directa de la fórmula de diferencias centrales para $f'(x)$.

Implementación de $f''(x)$

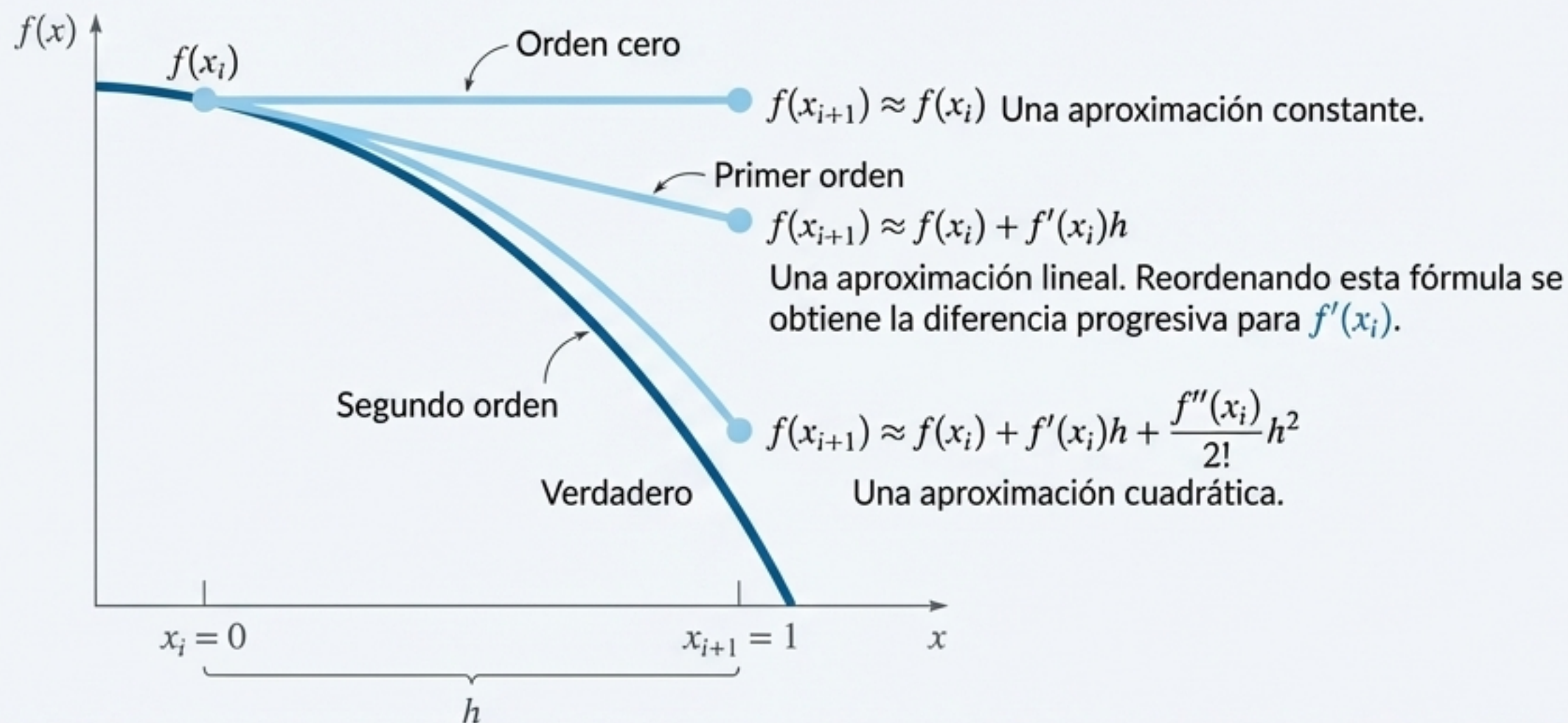
La función `segunda_derivada` es la implementación de la fórmula para $f''(x)$.

Control de Precisión

El parámetro `h` controla la distancia entre los puntos de muestreo; un `h` más pequeño generalmente conduce a una mayor precisión, hasta que se alcanzan los límites de la precisión numérica de la máquina.

El Fundamento Teórico: Conexión con la Serie de Taylor

Las fórmulas de diferencias divididas no son arbitrarias. **Se derivan directamente de la serie de Taylor**, que es una herramienta para aproximar una función cerca de un punto usando sus derivadas.



La Fuente de la Aproximación

Las diferencias finitas son esencialmente reordenamientos de la serie de Taylor truncada. El 'error de truncamiento' en la aproximación de la derivada proviene de los términos de orden superior que ignoramos.

Síntesis: El Poder de Construir Modelos a Partir de Datos

1. **El Problema:** Partimos de un conjunto de datos discretos, una situación común en cualquier disciplina técnica.
2. **La Solución:** Utilizamos el Polinomio de Lagrange para construir un modelo continuo, único y matemáticamente garantizado que se ajusta perfectamente a nuestros datos.
3. **Las Implicaciones:** Aprendimos a respetar los límites del modelo (Interpolación vs. Extrapolación) y a cuantificar su precisión mediante el análisis de error.
4. **La Extensión:** Ampliamos nuestro conjunto de herramientas para estimar no solo los valores de la función, sino también sus derivadas, utilizando diferencias divididas.

La interpolación y la diferenciación numérica no son solo ejercicios académicos. Son herramientas fundamentales y cotidianas para cualquier ingeniero o científico que necesite crear, analizar y simular sistemas del mundo real a partir de datos medidos. Dominarlas es dominar una parte esencial del modelado computacional.